Lab 03 - Divide & Conquer (Part 2)

Instruction:

- The objectives of this part (Part 2) are to:
  - Define the merge sort algorithm.
  - Generate its recurrence relation and derive its running time using the Master Theorem.
  - Verify and test the merge sort algorithm.
- The merge sort algorithm repeatedly divides an array into halves until the subarrays have length 1, and then sorts and merges them:

  `MergeSort(A, i, j)`

  01. if $i < j$, then
      01. $m \leftarrow \lfloor (i+j)/2 \rfloor$
      02. `MergeSort(A, i, m)`.
      03. `MergeSort(A, m + 1, j)`.
      04. `Merge(A, i, m, j)`.

  The core `Merge()` algorithm, which combines two adjacent sorted subarrays into one sorted subarray, is:

  `Merge(A, i, m, j)`

  01. declare an auxiliary array of length $j - i + 1$.
  02. until all elements of the subarrays $A[i, m]$ and $A[m + 1, j]$ have been copied into the auxiliary array, do
      01. compare the current head elements of the two subarrays, and copy the smaller one into the current slot of the auxiliary array.
      02. increment the current position in the chosen subarray and in the auxiliary array.
  03. copy the auxiliary array into the subarray $A[i, j]$.

  Note that this algorithm sorts A in ascending order.
- Submit the modified `.h` file from Part 1 and a new `.cpp` source file.
- Your files may include only the following libraries: `iostream`, `string`, `vector`, `ctime`, and `cstdlib`. No others are permitted.
- Submit your files to your GitHub repository, inside a directory named `Lab03`.
- Cheating of any kind is prohibited and will not be tolerated.
- **Violating or failing to follow any of these rules will result in an automatic zero (0) for the lab.**

At the very beginning of your submission, add a comment containing your name and the date, to confirm you have read and understood the instructions above.

Grading

Task	Maximum Points	Points Earned
1	0.5	
2	0.5	
3	0.5	
4	0.5	
5	0.5	
Total	2.5	

Task 1

In the header file from Part 1, within the namespace *aal*, define the function:

```
template <class T>
void Merge(vector<T>& v, int i, int m, int j)
```

that merges the two adjacent sorted subarrays $v[i,\ m]$ and $v[m+1,\ j]$ into one sorted subarray $v[i,\ j]$, as described above.

Task 2

Directly below your definition of `Merge()`, add a comment block that does the following:

1. Generates its running-time table, stating its input size and assuming that each statement has a cost of 1.
2. Calculates its running-time function.

Show all of your work.

Task 3

In the header file, within the namespace *aal*, define the function:

```
template <class T>
void MergeSort(vector<T>& v, int i, int j)
```

that sorts the subarray $v[i,\ j]$ in ascending order using the merge sort algorithm described above.

Task 4

Directly below your definition of `MergeSort()`, add a comment block that does the following:

1. States the recurrence relation of `MergeSort()`, and explains the values of each of its components (a, b, and $f(n)$).
2. Derives its running time using the Master Theorem.

Show all of your work.

Task 5

In the `main` function, verify that your `MergeSort()` function is correct by completing the following steps:

1. Initialize an integer vector object to the consecutive integers from 1 to 40.
2. Shuffle the contents of the vector object by invoking `Shuffle()` with that object as its argument.
3. Display the vector object as a list enclosed in curly braces.
4. Sort the entire vector object by invoking `MergeSort()` with that object and the index range $[0, 39]$ as its arguments.
5. If the vector object is sorted correctly, display it as a list enclosed in curly braces; otherwise, display an error message.